

CERTIFICACIÓN UNIVERSITARIA



UNC



FCEyN



Manual para el
alumno

Práctica Profesional IV



WE WANT
SKILLS!

mundosE

Manual para el alumno: Práctica Profesional 4.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
PARTE 1: INTRODUCCIÓN A LOS CONTENEDORES.....	3
¿Qué son los contenedores?.....	3
Ventajas de los contenedores:.....	3
¿Qué es Docker?.....	3
Componentes principales de Docker:.....	3
¿Qué es Kubernetes?.....	3
Conceptos clave de Kubernetes:.....	3
PARTE 2: INSTALACIÓN Y CONFIGURACIÓN DE DOCKER.....	4
Paso 1: Preparación del sistema.....	4
Paso 2: Instalación de Docker.....	4
Paso 3: Configuración de permisos.....	4
Paso 4: Explorando comandos básicos de Docker.....	5
Gestión de imágenes:.....	5
Gestión de contenedores:.....	5
Paso 5: Trabajando con un servidor web.....	5
Gestionar el contenedor:.....	6
Paso 6: Creando una imagen personalizada.....	6
Construir y ejecutar nuestra imagen:.....	8
PARTE 3: INTRODUCCIÓN A KUBERNETES.....	8
Paso 7: Instalación de Kubernetes con Minikube.....	8
Instalar kubectI (cliente de Kubernetes):.....	8
Instalar Minikube:.....	9
Paso 8: Iniciando tu primer cluster de Kubernetes.....	9
Paso 9: Conceptos básicos de Kubernetes.....	9
Explorando el cluster:.....	9
Creando tu primer pod:.....	10
Acceder al pod:.....	10
Paso 10: Trabajando con Deployments.....	10
Exponer el deployment como servicio:.....	11
Paso 11: Usando archivos YAML para Kubernetes.....	11
Aplicar los manifiestos:.....	13
Paso 12: Monitoreo y limpieza.....	13
RESOLUCIÓN DE PROBLEMAS COMUNES.....	14
Docker.....	14
Kubernetes.....	14
Bibliografía.....	15

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En él encontraremos un resumen de cada tema tratado en el encuentro, así como también recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Práctica Profesional 4

Objetivo del encuentro:

Realizar una práctica profesional donde se puedan desarrollar los contenidos de los temas vistos en las clases de contenedores, en la VM local.

Contenido a desarrollar:

- ✓ Instalar y configurar Docker en Ubuntu.
- ✓ Comprender los conceptos básicos de contenedores.
- ✓ Instalar y configurar herramientas básicas de Kubernetes.
- ✓ Ejecutar aplicaciones en contenedores.
- ✓ Entender las diferencias entre contenedores y máquinas virtuales.

PARTE 1: INTRODUCCIÓN A LOS CONTENEDORES

¿Qué son los contenedores?

Los contenedores son una tecnología de virtualización liviana que permite empaquetar aplicaciones junto con todas sus dependencias en un entorno aislado y portable. A diferencia de las máquinas virtuales tradicionales, los contenedores comparten el *kernel* del sistema operativo *host*, lo que los hace más eficientes en recursos.

Ventajas de los contenedores:

- **Portabilidad:** "Funciona en mi máquina" se convierte en "funciona en cualquier máquina".
- **Eficiencia:** Menor uso de recursos comparado con VMs completas.
- **Escalabilidad:** Fácil de escalar horizontalmente.
- **Consistencia:** Mismo comportamiento en desarrollo, pruebas y producción.
- **Rapidez:** Arranque e implementación más rápida.

¿Qué es Docker?

Docker es la plataforma de contenedores más popular del mundo. Permite crear, distribuir y ejecutar aplicaciones en contenedores de manera sencilla.

Componentes principales de Docker:

- **Docker Engine:** Motor que ejecuta los contenedores.
- **Docker Images:** Plantillas de solo lectura para crear contenedores.
- **Docker Containers:** Instancias ejecutables de las imágenes.
- **Dockerfile:** Archivo de texto con instrucciones para construir imágenes.
- **Docker Hub:** Registro público de imágenes de Docker.

¿Qué es Kubernetes?

Kubernetes (K8s) es un sistema de orquestación de contenedores que automatiza el despliegue, escalado y gestión de aplicaciones contenerizadas.

Conceptos clave de Kubernetes:

- **Cluster:** Conjunto de nodos que ejecutan aplicaciones contenerizadas.
- **Pod:** Unidad básica de despliegue que contiene uno o más contenedores.
- **Deployment:** Define cómo se despliegan y actualizan los pods.
- **Service:** Proporciona acceso de red a un conjunto de pods.
- **Node:** Máquina worker que ejecuta pods.

PARTE 2: INSTALACIÓN Y CONFIGURACIÓN DE DOCKER

Paso 1: Preparación del sistema

Primero, vamos a actualizar el sistema y preparar las dependencias necesarias.

```
# Actualizar repositorios  
sudo apt update
```

```
# Verificar la versión de Ubuntu  
lsb_release -a
```

¿Qué estamos haciendo?

- `apt update` descarga la información más reciente de los paquetes disponibles.
- `lsb_release -a` muestra información detallada sobre la distribución Ubuntu.

Paso 2: Instalación de Docker

Vamos a instalar Docker usando el método más sencillo desde los repositorios oficiales de Ubuntu:

```
# Instalar Docker desde los repositorios de Ubuntu  
sudo apt install -y docker.io
```

```
# Verificar la instalación  
docker --version
```

 **PUNTO DE CONTROL 1:** Deberías ver algo como "Docker version 27.x.x".

Paso 3: Configuración de permisos

Para usar Docker sin sudo constantemente, necesitamos agregar nuestro usuario al grupo docker:

```
# Agregar usuario al grupo docker  
sudo usermod -aG docker $USER
```

```
# Aplicar cambios al grupo (o reiniciar sesión)  
newgrp docker
```

```
# Verificar que podemos ejecutar Docker sin sudo  
docker run hello-world
```

¿Qué está pasando?

- El comando `hello-world` descarga una imagen de prueba y ejecuta un contenedor.
- Si ves el mensaje "Hello from Docker!", ¡Docker está funcionando correctamente!

Paso 4: Explorando comandos básicos de Docker

Gestión de imágenes:

Listar imágenes descargadas

`docker images`

Buscar imágenes en Docker Hub

`docker search nginx`

Descargar una imagen específica

`docker pull nginx:alpine`

Gestión de contenedores:

Listar contenedores en ejecución

`docker ps`

Listar todos los contenedores (activos e inactivos)

`docker ps -a`

Ejecutar un contenedor interactivo

`docker run -it ubuntu:22.04 bash`

Dentro del contenedor Ubuntu, prueba:

Estás dentro del contenedor

`whoami`

`hostname`

`cat /etc/os-release`

`exit` # Para salir del contenedor

Paso 5: Trabajando con un servidor web

Vamos a crear nuestro primer servicio real:

Ejecutar nginx en segundo plano
docker run -d --name mi-nginx -p 8080:80 nginx:alpine

Verificar que está ejecutándose
docker ps

Verificar los logs
docker logs mi-nginx

Probar el servidor web
curl http://localhost:8080

¿Qué significan estos parámetros?

- **-d**: ejecuta en segundo plano (daemon)
- **--name mi-nginx**: asigna un nombre al contenedor
- **-p 8080:80**: mapea el puerto 8080 del host al puerto 80 del contenedor

Gestionar el contenedor:

Detener el contenedor
docker stop mi-nginx

Reiniciar el contenedor
docker start mi-nginx

Eliminar el contenedor (primero hay que detenerlo)
docker stop mi-nginx
docker rm mi-nginx

Paso 6: Creando una imagen personalizada

Vamos a crear nuestra propia imagen Docker con una aplicación web simple:

Crear directorio para nuestro proyecto
mkdir mi-app-web
cd mi-app-web

Crear una página web simple
cat > index.html << 'EOF'

<!DOCTYPE html>

<html>

```

<head>

  <title>Mi Primera App Docker</title>

  <style>

    body { font-family: Arial; text-align: center; background: #f0f0f0; }

    .container { margin: 50px auto; padding: 20px; background: white;
max-width: 600px; border-radius: 10px; }

  </style>

</head>

<body>

  <div class="container">

    <h1>🐳 Mi Primera Aplicación Docker</h1>

    <p>¡Felicitaciones! Has creado tu primera imagen Docker
personalizada.</p>

    <p>Fecha y hora: <span id="datetime"></span></p>

    <script>

      document.getElementById('datetime').textContent = new
Date().toLocaleString();

    </script>

  </div>

</body>

</html>

```

EOF

```

# Crear el Dockerfile
cat > Dockerfile << 'EOF'

```

```

# Usar nginx como imagen base

```


FROM nginx:alpine

Copiar nuestro HTML al directorio web de nginx
COPY index.html /usr/share/nginx/html/

Exponer el puerto 80
EXPOSE 80

nginx se ejecuta automáticamente al iniciar el contenedor
EOF

Construir y ejecutar nuestra imagen:

Construir la imagen
docker build -t mi-app:v1.0 .

Verificar que se creó
docker images

Ejecutar nuestra aplicación
docker run -d --name mi-aplicacion -p 8081:80 mi-app:v1.0

Probar la aplicación
curl http://localhost:8081

 **PUNTO DE CONTROL 2:** Deberías poder ver tu página web personalizada.

PARTE 3: INTRODUCCIÓN A KUBERNETES

Paso 7: Instalación de Kubernetes con Minikube

Minikube es una herramienta que ejecuta un cluster de Kubernetes localmente en una sola máquina.

Instalar kubectl (cliente de Kubernetes):

Descargar kubectl
curl -LO
"https://storage.googleapis.com/kubernetes-release/release/v1.28.0/bin/
linux/amd64/kubectl"

Dar permisos de ejecución
chmod +x kubectl

Mover a directorio del PATH
sudo mv kubectl /usr/local/bin/

Verificar instalación
kubectl version --client

Instalar Minikube:

Descargar Minikube
curl -Lo minikube
<https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64>

Dar permisos de ejecución
chmod +x minikube

Mover a directorio del PATH
sudo mv minikube /usr/local/bin/

Verificar instalación
minikube version

Paso 8: Iniciando tu primer cluster de Kubernetes

Iniciar Minikube con Docker como driver
minikube start --driver=docker

Verificar el estado
minikube status

Verificar los nodos del cluster
kubectl get nodes

Ver información detallada del cluster
kubectl cluster-info

 **PUNTO DE CONTROL 3:** Deberías ver un nodo llamado "minikube" en estado "Ready".

Paso 9: Conceptos básicos de Kubernetes

Explorando el cluster:

Ver todos los pods del sistema
kubectl get pods -A

Ver los namespaces
kubectl get namespaces

Ver los servicios
kubectl get services

Creando tu primer pod:

Crear un pod con nginx
kubectl run mi-pod --image=nginx:alpine

Ver el pod
kubectl get pods

Ver detalles del pod
kubectl describe pod mi-pod

Ver logs del pod
kubectl logs mi-pod

Acceder al pod:

Ejecutar comandos dentro del pod
kubectl exec -it mi-pod -- sh

Dentro del pod:
hostname
ps aux
exit

Paso 10: Trabajando con Deployments

Los Deployments son la forma recomendada de gestionar aplicaciones en Kubernetes:

Crear un deployment
kubectl create deployment nginx-demo --image=nginx:alpine

Ver el deployment
kubectl get deployments

Ver los pods creados por el deployment
kubectl get pods

Escalar el deployment
kubectl scale deployment nginx-demo --replicas=3

Verificar que ahora hay 3 pods
kubectl get pods

Exponer el deployment como servicio:

Crear un servicio para exponer el deployment

kubectl expose deployment nginx-demo --type=NodePort --port=80

Ver el servicio

kubectl get services

Obtener la URL del servicio

minikube service nginx-demo --url

Nota: Anota la URL que muestra el comando anterior para acceder al servicio.

Paso 11: Usando archivos YAML para Kubernetes

En lugar de comandos, podemos definir nuestros recursos usando archivos YAML:

Crear un directorio para nuestros manifiestos

mkdir k8s-manifests

cd k8s-manifests

Crear un deployment YAML

cat > mi-app-deployment.yaml << 'EOF'

apiVersion: apps/v1

kind: Deployment

metadata:

name: mi-aplicacion

labels:

app: mi-aplicacion

spec:

replicas: 2

selector:

matchLabels:

app: mi-aplicacion

template:

metadata:

labels:

app: mi-aplicacion

spec:

containers:

- name: web

image: nginx:alpine

ports:

- containerPort: 80

resources:

requests:

memory: "64Mi"

cpu: "250m"

limits:

memory: "128Mi"

cpu: "500m"

EOF

Crear un servicio YAML

cat > mi-app-service.yaml << 'EOF'

apiVersion: v1

kind: Service

metadata:

name: mi-aplicacion-service

spec:

selector:

app: mi-aplicacion

ports:

- protocol: TCP

port: 80

targetPort: 80

nodePort: 30080

type: NodePort

EOF

Aplicar los manifiestos:

Aplicar el deployment

kubectl apply -f mi-app-deployment.yaml

Aplicar el servicio

kubectl apply -f mi-app-service.yaml

Verificar que todo se creó correctamente

kubectl get deployments,pods,services

Paso 12: Monitoreo y limpieza

Ver el estado de todos los recursos

kubectl get all

Describir un recurso específico

kubectl describe deployment mi-aplicacion

Ver eventos del cluster

kubectl get events --sort-by=.metadata.creationTimestamp

Eliminar recursos

kubectl delete -f mi-app-deployment.yaml

kubectl delete -f mi-app-service.yaml

kubectl delete pod mi-pod

kubectl delete deployment nginx-demo

kubectl delete service nginx-demo

Detener Minikube

minikube stop

RESOLUCIÓN DE PROBLEMAS COMUNES

Docker

1. **Error de permisos:** `sudo usermod -aG docker $USER && newgrp docker`
2. **Sin espacio:** `docker system prune -a`
3. **Contenedor no inicia:** `docker logs <nombre_contenedor>`

Kubernetes

4. **Pod en Pending:** `kubectl describe pod <nombre_pod>`
5. **Servicio no accesible:** `kubectl get endpoints`
6. **Minikube no inicia:** `minikube delete && minikube start --driver=docker`

Bibliografía

- **Docker** © 2013-2025. Docker Hub – <https://docs.docker.com/>
- **Play with Kubernetes** © 2017 - 2023. Kubernetes Playground.
<https://labs.play-with-k8s.com/>
- **The Kubernetes Authors** © (2025). Cheat Sheets:
<https://kubernetes.io/docs/reference/kubectl/cheatsheet/>